

Maragal_8 - Introduction

Background and explanation of the reproducibility experiment

1. What this experiment is for

This folder contains the reproducibility material for the Maragal_8 experiment used in the dissertation. The experiment uses a large sparse matrix from the SuiteSparse Matrix Collection as a fixed test problem for comparing PCG, LSQR and LSMR. The purpose is to compare the methods under the same matrix, the same right-hand side and the same regularisation sweep.

This is not a reconstruction of the original New York Power Authority calculation. The supplied program uses the Maragal_8 matrix as a realistic large sparse test case and constructs its own deterministic right-hand side so that the experiment can be reproduced without application-specific measurements.

2. The Maragal_8 problem

Maragal_8 belongs to the NYPA group of the SuiteSparse Matrix Collection and is described by the collection as a rank-deficient least-squares problem associated with D. Maragal and the New York Power Authority. It is rectangular, sparse and numerically rank deficient. These properties make it a useful test for iterative methods and for avoiding explicit formation of large normal-equation matrices.

Property	Value
Collection	SuiteSparse Matrix Collection, NYPA
Matrix	Maragal_8
Rows, m	33,212
Columns, n	75,077
Nonzeros	1,308,415
Numerical rank	15,343
Null-space dimension	17,869
Problem type	Rank-deficient least-squares
Data type	Real, rectangular, sparse

In the notation used in the code, A is an m by n matrix, x has n entries and b has m entries. Here m = 33,212 and n = 75,077, so there are more unknowns than equations. The matrix is also rank deficient, meaning that the least-squares problem does not have a unique unregularised solution.

3. The right-hand side used in the benchmark

The program deliberately constructs the right-hand side rather than reading an observed application vector. It first creates a vector of ones of length n and then calculates $b = A @ \text{ones}$. Every solver therefore receives exactly the same b for a given matrix.

This detail must not be changed when reproducing the dissertation benchmark. Replacing b with an arbitrary vector or with an original application right-hand side would create a different experiment.

4. The mathematical problem

For each value of lambda, the code compares the regularised least-squares objective:

```
minimise ||A x - b||^2 + lambda ||x||^2
```

Lambda controls the amount of regularisation. The supplied program tests seven values: 1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 1e-1 and 1.0. The complete sweep is retained so that the comparison does not depend on selecting one convenient value after seeing a result.

5. The three solvers

Method	Implementation in the supplied program
PCG	CG on $(A^T A + \lambda I)x = A^T b$. $A^T A$ is never explicitly formed. The product is applied as $A^T (A v)$, with a Jacobi diagonal preconditioner.
LSQR	Damped LSQR on A and b, with $\text{damp} = \sqrt{\lambda}$.
LSMR	Damped LSMR on A and b, with $\text{damp} = \sqrt{\lambda}$.

The matrix-free detail is important. The PCG implementation does not build a separate large matrix for $A^T A$. Instead, the iterative method asks for products with $A^T A$ and the program evaluates them using two sparse matrix-vector operations. This is one of the reasons the experiment is practical for a matrix of this size.

6. What is measured

For every solver, lambda and repeat, the program records runtime, iteration count, residuals, objective value, convergence status, preconditioner information and whether the calculation was matrix-free.

Measure	Meaning
Time (s)	Wall-clock time measured around the solver call.
Iterations	Iterations reported by the solver, or counted by the PCG callback.
Residual	Norm of $A x - b$.
Relative Residual	Residual divided by the norm of b.
Normal Equation Residual	Norm of $A^T (A x - b) + \lambda x$.
Objective	Residual squared plus lambda times the squared norm of x.
Converged	Convergence status reported or derived by the supplied code.

7. Files in this folder

File	Purpose
Maragal_8.py	Runs the benchmark and writes the CSV results.
plot_Maragal_8.py	Reads the generated results and creates plots.
Maragal_8_Introduction.pdf	Explains the experiment.
Maragal_8_Run_Instructions.pdf	Explains how to reproduce it.
Maragal_8_Requirements_and_Restrictions.pdf	Records resource and methodological requirements.